



UNIVERSIDADE FEDERAL
DO ESPÍRITO SANTO

Centro Tecnológico
Departamento de Informática

Prof. Vítor E. Silva Souza

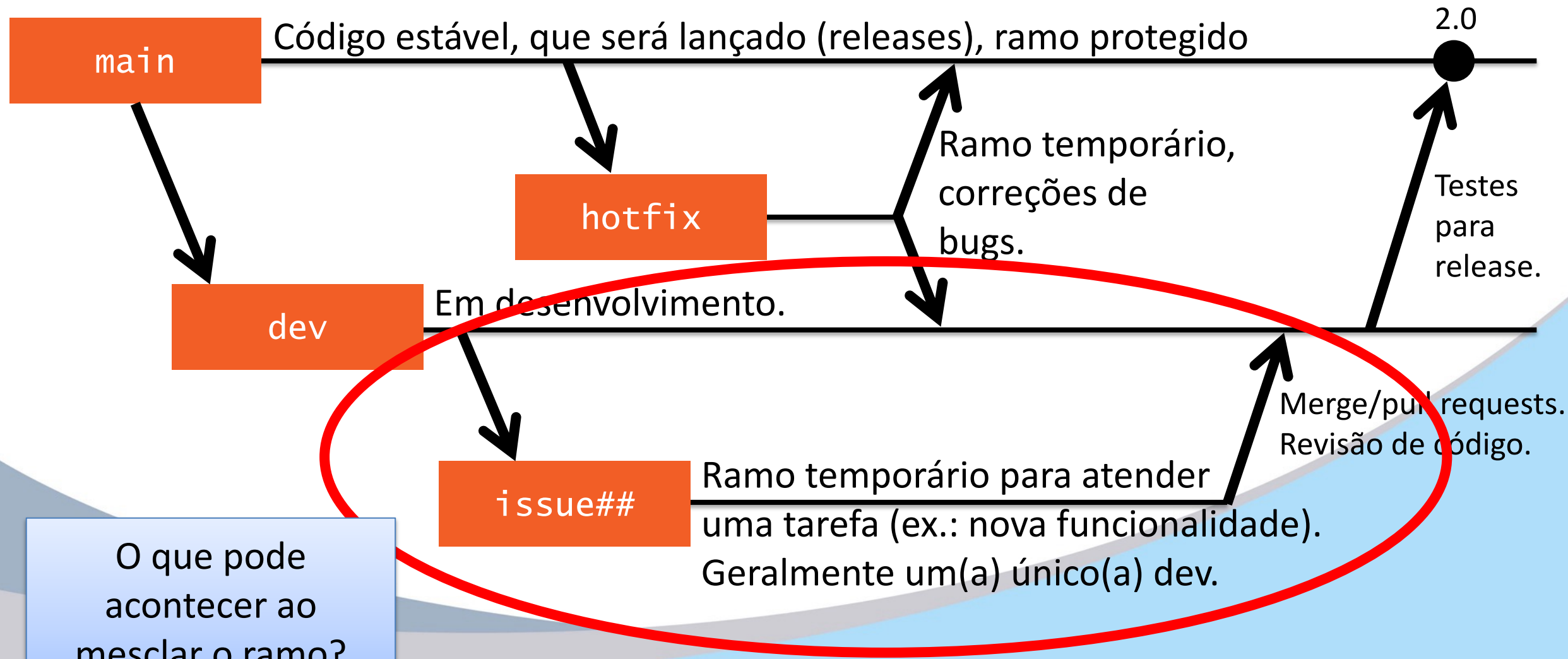
<http://www.inf.ufes.br/~vitorsouza>

CI & CD: Integração e Implantação Contínuas



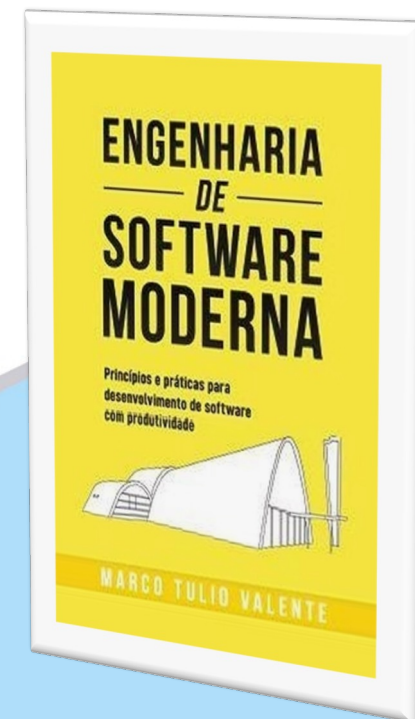
This material is licensed under the Creative Commons license Attribution-ShareAlike 4.0 International: <http://creativecommons.org/licenses/by-sa/4.0/>.

Relembrando Git Flow



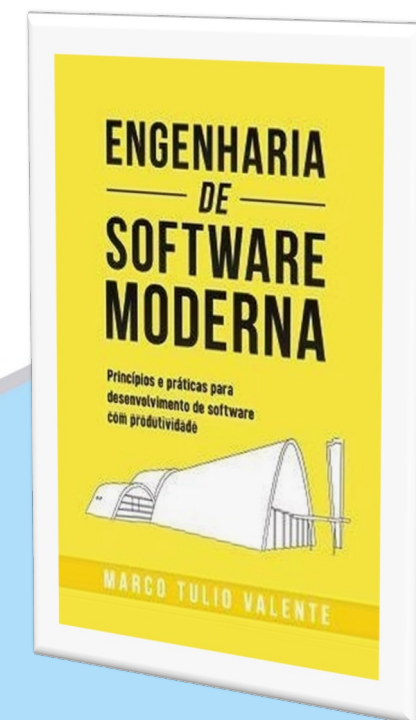
Motivação

- Ao mesclar um ramo de funcionalidade (F) num ramo principal (P), pode acontecer:
 - *Algo que foi usado em F foi modificado em P;*
 - *Algo modificado em F ao mesclar em P causa erros em outras partes do código;*
- Quanto mais “longos” os ramos F maior a chance destes problemas ocorrerem;
- Quanto mais problemas, mais dolorosa será a mesclagem / integração.



Integração Contínua

- *Continuous Integration* ou CI;
- Integrar códigos com mais frequência, ou seja, integrar de forma contínua;
 - *Integrações pequenas;*
 - *Menos conflitos;*
 - *Testes mais frequentes.*



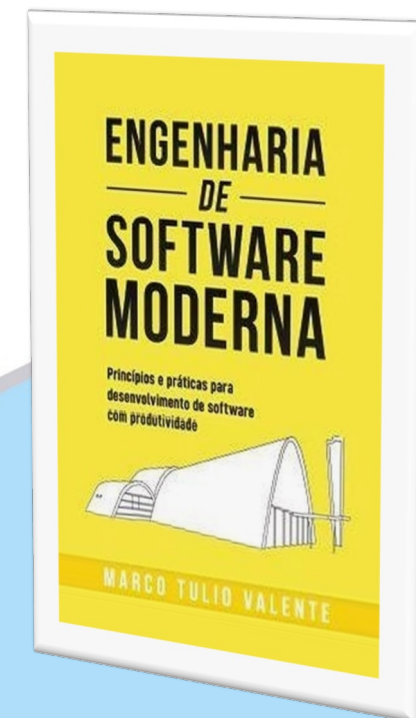
“Você deve integrar e testar o seu código em intervalos menores do que algumas horas. Programação em times não é um problema do tipo dividir-e-conquistar. Na verdade, é um problema que requer dividir, conquistar e integrar. A duração de uma tarefa de integração é imprevisível e pode facilmente levar mais tempo do que a tarefa original de codificação. Assim, quanto mais tempo você demorar para integrar, maiores e mais imprevisíveis serão os custos.”



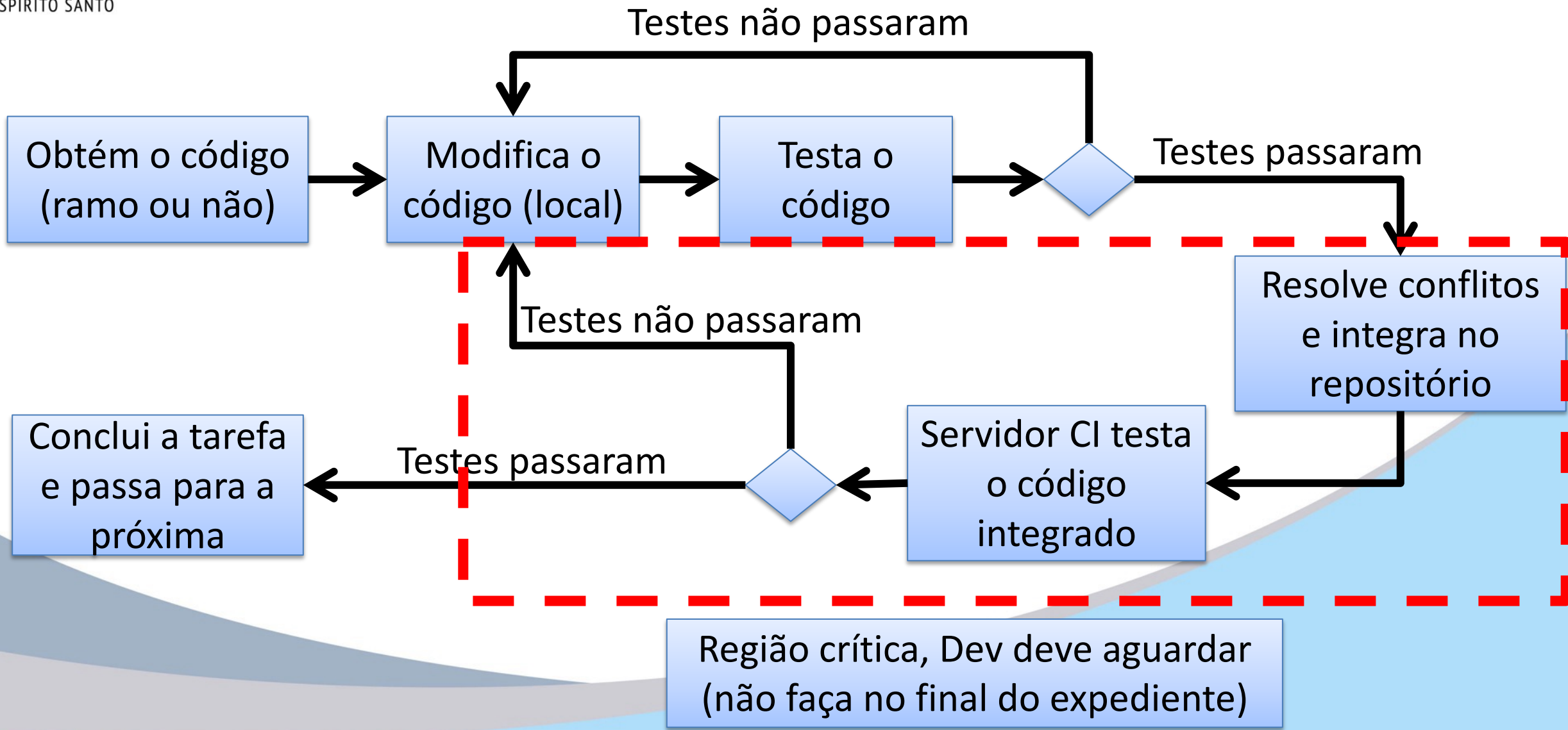
Kent Beck
Criador do XP e do TDD

Boas práticas para uso de CI

- Construção (*build*) automatizada;
- Testes automatizados;
- Servidores de integração contínua;
- Ramos de curta duração ou uso de *trunk-based development* (TBD);
- (Opcional) Programação em Pares.



Fluxo de desenvolvimento com CI



Teste + integração + teste

- Testes locais: funciona na máquina local (é o básico);
- Integração: se algo mudou no ramo principal e está causando erro de construção, corrigir;
- Testes CI: se algo mudou no ramo principal e poderá causar erros em tempo de execução, um bom conjunto de testes automatizados pode capturar.

Pergunta: quando CI é indicada e quando não é?

Implantação contínua

- *Continuous Deployment* ou CD;
- Extensão da CI (faz todos os passos da CI);
- Adiciona:
 - *Testes mais extensos (integração, interface, desempenho);*
 - *Implantação automática **em produção**.*

Pode-se fazer implantação automática em ambientes de teste/homologação, só não é considerado CD.

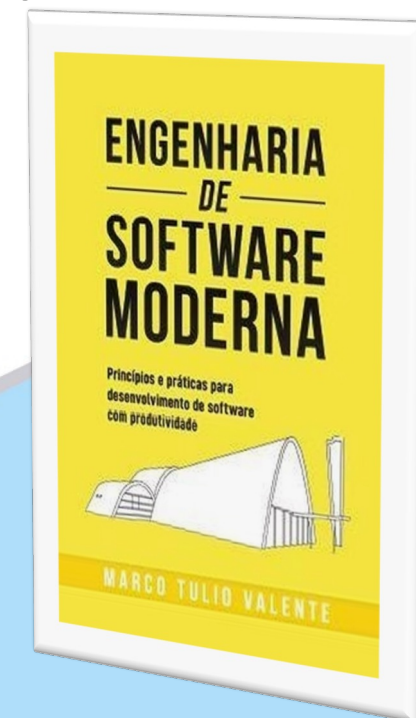
Pergunta: quando é indicada e quando não é?



Entrega Contínua

- *Continuous Delivery;*
- *“Versão mais fraca” de CD:*
 - *Intervenção humana de quando a implantação ocorrerá;*
 - *Pode ser feita também em intervalos de tempo definidos.*

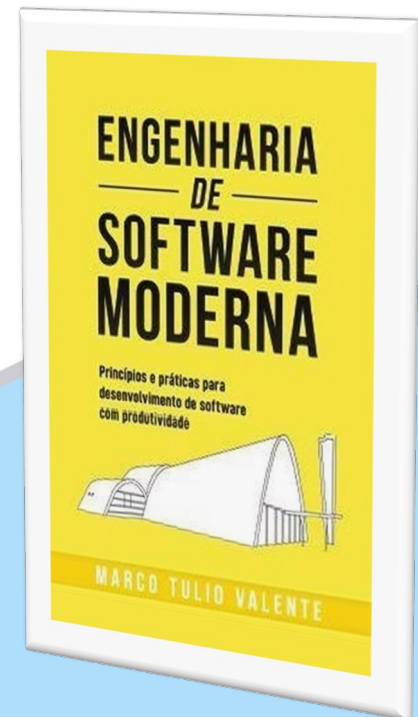
Pergunta: quando é indicada e quando não é?



Feature flags

- Com mudanças bem pequenas integradas em produção, como impedir usuários finais de ver coisas incompletas?
- Código parcial é integrado e implantado continuamente, porém desabilitado;
 - *Variável Feature Flag ou Feature Toggle +*
 - *Condicional (if) que habilita ou não o código;*
- Também podem ser usadas para fazer “release canário” e testes A/B.

Há uma série de soluções prontas para *feature flagging*



Exemplo CI: Lab Maven 02

```
name: Java CI with Maven
on:
  push:
    branches: [ development ]
  pull_request:
    branches: [ development ]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout source
        uses: actions/checkout@v4
      - name: Set up JDK 21
        uses: actions/setup-java@v4
```

 .github/workflows/development.yml

```
with:
  java-version: '21'
  distribution: 'temurin'
  cache: maven
- name: Build with Maven
  run: mvn -B package --file
pom.xml
```



Roda os testes e avisa
se houver erros.

Exemplo CI/CD*: Marvin @ LabES



Repositório Git +
Servidor CI/CD

[https://gitlab.labes.inf.ufes.br/
marvin/marvin](https://gitlab.labes.inf.ufes.br/marvin/marvin)



Runner CI/CD

[https://dev.labes.inf.ufes.br/
marvin/](https://dev.labes.inf.ufes.br/marvin/)

Instruções de instalação: [https://gitlab.labes.inf.ufes.br/labes/catalogo/-
/wikis/ferramentas/gitlab-pipelines](https://gitlab.labes.inf.ufes.br/labes/catalogo/-/wikis/ferramentas/gitlab-pipelines)



* Não é de fato CD pois a implantação é em ambiente de testes/homologação. Mas poderia ser. 😊

Componentes da solução CI/CD do LabES

- Docker: executa os sistemas implantados em contêineres (ficam isolados uns dos outros);
- PostgreSQL: armazena os dados dos sistemas implantados (decidiu-se não usar contêineres para ele);
- Apache HTTP Server: redireciona URLs amigáveis para a URL dos sistemas nos contêineres;
- Portainer: interface amigável via Web para gerenciar contêineres Docker.

Variáveis no servidor CI/CD

- STAGING_SERVER_HOSTNAME : dev.labes.inf.ufes.br
- STAGING_SERVER_USER : deployer
- STAGING_SERVER_PSQL_USERNAME : labes
- STAGING_SERVER_PSQL_PASSWORD : (defina uma senha para este usuário e insira-a aqui)

CI/CD Variables </> 5

↑ Key

STAGING_SERVER_HOSTNAME 

***** 





Protected Masked

STAGING_SERVER_ID_FILE 

***** 





File Protected

STAGING_SERVER_PSQL_PASSWORD 

***** 





Protected Masked

STAGING_SERVER_PSQL_USERNAME 

***** 





Protected

STAGING_SERVER_USER 

***** 





Protected Masked

Script CI/CD do Marvin (1)

 .gitlab-ci.yml

```
# Four stages in this pipeline: run
# tests with Maven, build the
# application with Maven, publish image
# in Docker registry, deploy using
# Docker.
```

stages:

- test
- build
- publish
- deploy

```
# Continua nos próximos slides...
```

✓ MARVIN

> .idea

> docs

> META-INF

> **src**

> target

 .gitignore

 .gitlab-ci.yml

 docker-compose-gitlab.yml

 Dockerfile

 LICENSE.txt

 pom.xml

 README.md

Script CI/CD do Marvin (2)

 .gitlab-ci.yml

```
variables:
  # Maven options.
  MAVEN_OPTS: "..."/>

```

Valor omitido por ser muito extenso.

```

  # Provide tags for the latest commit and for each commit.
  TAG_LATEST: $CI_REGISTRY_IMAGE/$CI_COMMIT_REF_NAME:latest
  TAG_COMMIT:
$CI_REGISTRY_IMAGE/$CI_COMMIT_REF_NAME:$CI_COMMIT_SHORT_SHA

  # Directory in which to perform the deployment.
  DEPLOY_DIR: ~/gitlab-
deployments/$CI_PROJECT_NAMESPACE/$CI_PROJECT_NAME

  # Name of the database to be created, if doesn't exist.
  DATABASE_NAME: marvin

```

Script CI/CD do Marvin (3)

 .gitlab-ci.yml

```
# Cache downloaded dependencies and plugins between Maven
# builds (test stage).
cache:
  paths:
    - .m2/repository

# JOB: test (test stage). For every commit, verify the
# project (e.g., run unit tests).
test:
  image: maven:3.9.5-eclipse-temurin-21
  stage: test
  script:
    - mvn $MAVEN_OPTS verify
```

Script CI/CD do Marvin (4)

 .gitlab-ci.yml

```
# JOB: build (build stage). Build the Web app with Maven.
build:
  image: maven:3.9.5-eclipse-temurin-21
  stage: build
  script:
    # The sed commands replace the development-stage
    # DB config with the one used in the staging server.
    - sed -i -e ...
    - sed -i -e ...
    - mvn $MAVEN_OPTS -Dmaven.test.skip package
  artifacts:
    paths:
      - target/*.war

# Job build should only run when the branch is the default,
# i.e., on merge requests being merged into main.
only:
  variables:
    - $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
```

Script CI/CD do Marvin (5)

 .gitlab-ci.yml

```
# JOB: publish (publish stage). Build the docker image.
publish:
  image: docker:20-dind
  stage: publish
  tags:
    - deployment
  variables:
    ...
  services:
    ...
  script:
    - docker login -u $CI_REGISTRY_USER -p ...
    - docker build -t $TAG_COMMIT -t $TAG_LATEST .
    - docker push $TAG_COMMIT
    - docker push $TAG_LATEST
# Same as before
only:
  variables:
    - $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
```

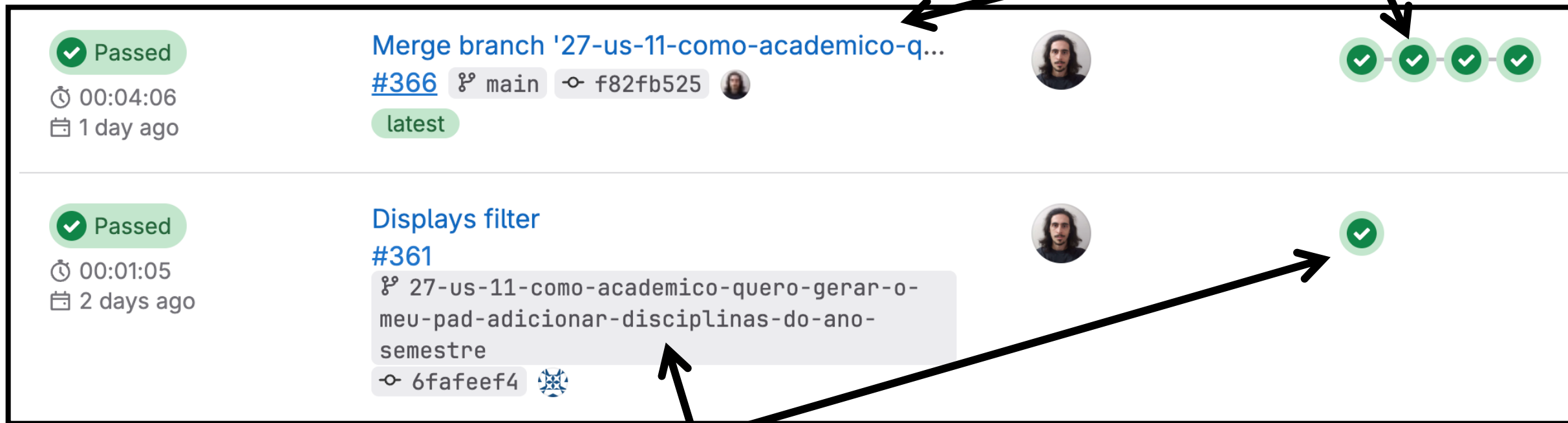
Script CI/CD do Marvin (6)

 .gitlab-ci.yml

```
# JOB: deploy (deploy stage). Deploys the application.
deploy:
  image: alpine:latest
  stage: deploy
  tags:
    - deployment
  script:
    # 1. Creates a deploy script and an .env file with
    # variables to be used in docker-compose.yml.
    - ...
    # 2. Installs SSH and copies the files to a temporary
    # folder in the staging server.
    - ...
    # 3. Runs the deploy script on the server.
    - ...
# Same as before
only:
  variables:
    - $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
```

CI/CD do Marvin no GitLab

Mesclagem no ramo
principal: CI + CD



The screenshot displays a GitLab CI/CD pipeline with two jobs:

- Job 1: Merge branch '27-us-11-como-academico-q...'**
 - Status: Passed
 - Duration: 00:04:06
 - Time: 1 day ago
 - Branch: `main`
 - Commit: `f82fb525`
 - Label: `latest`
- Job 2: Displays filter**
 - Status: Passed
 - Duration: 00:01:05
 - Time: 2 days ago
 - Branch: `27-us-11-como-academico-quero-gerar-o-meu-pad-adicionar-disciplinas-do-ano-semester`
 - Commit: `6fafeef4`

An arrow points from the 'Merge branch' job to the 'Displays filter' job, indicating a merge action.

Commit no ramo de
funcionalidade: apenas CI

Marvin implantado automaticamente

Container list ↻

Containers

Q Search...

☐	Name ↓↑	State ↓↑	Filter ▾	Quick Actions	Stack ↓↑	Image ↓↑
☐	portainer	running		📄 ⓘ ⌵ 🔗	-	portainer/portainer-
☐	wildfly-marvin	running		📄 ⓘ ⌵ 🔗	marvin	gitlab.labes.inf.ufes

Marvin

dev.labes.inf.ufes.br/marvin/login.xhtml

☆

Anônimas (2)

⋮

Marvin @ UFES

Identifique-se para poder utilizar o Marvin

Por favor digite o seu endereço de e-mail

Por favor digite aqui a sua senha

Entrar

Esqueci minha senha

Quero criar uma conta

Maio 2024

Projeto Integrado I - CI & CD

23

Para saber mais

